NFL Pick'em Tracker — Build Spec

Written 2026-09-02. Handoff brief for building this as a website. Every technical detail below was confirmed against a real sample file, not guessed.

1. What this is

Three people (Patrick, Dalton, Hunter) run a season-long NFL "pick'em against the spread" contest. Every week, a commissioner sends a PDF of that week's point spreads. Each of the 3 people picks a side (against the spread) for every game. **There are always exactly 3 votes and 2 possible picks per game, so a majority (2-1 or 3-0) always exists — no ties are possible.** That majority is the group's official pick for the game.

The tool needs to:

- Take the weekly PDF of lines and turn it into a structured list of games/spreads, ideally automatically (parsing the PDF) rather than manual re-typing.
- Let all three people submit their own pick for every game, from wherever they are (this must be a real website reachable remotely — not a local-only app — since all three pick independently).
- Automatically compute the majority/group pick per game once all three have picked.
- Once final scores are known, grade each game against the spread (ATS winner), and score both:
  - the **group's** official record (season + weekly), and
  - each of the **3 individuals'** own personal accuracy (did their own pick match the ATS result, independent of the group majority).

2. The weekly PDF format (confirmed from a real sample, "Week 7.pdf")

Structure, top to bottom:

- Title line: `Week N`
- Day-of-week section headers — `Thursday` / `Sunday` / `Monday` in the sample, but this can vary week to week (e.g. a Friday international game, a Saturday late-season game). Don't hardcode an exact day list.
- Each day section is a grid of bordered boxes, roughly two boxes per row, one box per game.
- Each game box contains:
  - An optional note line above everything else, for special games — e.g. `London` plus a kickoff time like `8:30 AM` for an early/international game. Not present on most games.

- A point-spread number, e.g. `5 1/2`, `11 1/2`, `7` (whole numbers appear with no fraction).
- Below that, the matchup line: `Away Team   at   Home Team`.

- A `Teams on Bye:` line near the bottom listing bye-week teams, comma-separated.
- A blank `Name` signature line at the very bottom — this is a leftover from when the sheet used to be printed and filled out by hand. It's not meaningful data and can be ignored.

> **Critical parsing detail:** the spread number is positioned horizontally (left vs. right) directly above whichever team name it applies to — *that position is what tells you which team is favored*. For example, in the box for "LA Rams at Jacksonville", the number "3" sits above "LA Rams" (the away team) — meaning the Rams are favored by 3, even though they're on the road (they're playing in London, which is what the extra note above Jacksonville's side communicates). Plain top-to-bottom, left-to-right text extraction (e.g. a naive PDF-to-text library like PyPDF2) throws away this horizontal alignment and cannot tell you which team is favored. **Whatever PDF-parsing approach is used must read word-level bounding boxes/coordinates** (in Python, the `pdfplumber` library's `page.extract_words()` gives each word's `x0`/`x1`/`top`/`bottom`) so the spread number's x-position can be compared against each team name's x-range within the same box, to determine which team it sits above.

Team names in the PDF are short/informal city or market names, e.g. `Pittsburgh`, `Cincinnati`, `LA Rams`, `NY Giants`, `LA Chargers`, `Green Bay`, `Washington`, `New Orleans`. If scores are pulled from an external API (see below), a mapping table from these PDF-style names to whatever team identifier that API uses will be needed — build it by hand, and validate it against a few more real weekly PDFs since only a handful of team names have actually been confirmed so far.

3. Final scores, for grading

A free, no-API-key NFL scoreboard endpoint was tested live and confirmed working:

```
GET
https://site.api.espn.com/apis/site/v2/sports/football/nfl/scoreboard?week={week}&seasontype=2&date
```

`seasontype=2` is the regular season (this project is regular season only — Weeks 1-18, no playoffs). The JSON response has an `events[]` array; each event has `competitions[0].competitors[]`, where each competitor has:

- `homeAway`: `"home"` or `"away"`
- `score`: final score (string)
- `team.displayName` (e.g. `"Cincinnati Bengals"`), `team.abbreviation` (e.g. `"CIN"`)

and the event's `status.type.completed` (boolean) / `status.type.name` (`"STATUS_FINAL"` when done) tells you whether it's safe to grade yet — don't grade a game that isn't final. Confirmed live:

week=7, dates=2025 returned the exact same Thursday game from the sample PDF (Pittsburgh at Cincinnati), final score Bengals 33 – Steelers 31, matching real life.

Alternative: if pulling scores automatically is more trouble than it's worth, manual entry of final scores (typed in by whoever) is a perfectly fine fallback — the grading math is the same either way, only the score's source changes.

4. Scoring / grading rules

- **ATS winner per game**: let `margin = (favorite's actual score − underdog's actual score) − spread`. If `margin > 0`, the favorite covered; if `margin < 0`, the underdog covered; if `margin == 0`, it's a **push** (recommended: treat a push as void — no win or loss for anyone, tracked separately).
- **Group's majority pick per game**: whichever team got 2 or 3 of the 3 individual picks.
- **Group record**: compare the majority pick to the ATS winner, tally wins/losses/pushes, both per-week and running season-to-date.
- **Individual record per person**: compare that person's own pick directly to the ATS winner (regardless of what the group majority was), same weekly + season tally.
- If a game hasn't gotten all 3 picks in before it needs to be graded, exclude it from the group record until all 3 have picked (this shouldn't normally happen if everyone picks before games start).

5. Access / who uses it

- Three known users (Patrick, Dalton, Hunter) — no need for full user accounts/registration, but each person does need to be able to identify themselves and submit/see their own picks distinctly from the other two.
- Needs to be reachable remotely by all three (a real hosted website, not something that only runs on one person's machine).
- A single shared passcode gate (one password for all three, not per-person credentials) is sufficient to keep the site from being wide open to strangers — this doesn't need to be more elaborate than that.

6. Desired flow (suggested, not prescriptive)

1. **Upload week**: upload that week's PDF; it parses into a list of games (matchup, spread, favored team); show the parsed result for a quick human sanity-check/correction before saving, since PDF layout quirks can vary week to week; save it as that week's official lines.

2. **Make picks**: each person selects who they are, then picks a side for every game that week; can revise their pick anytime before it's graded.
3. **Standings**: pull in / enter final scores; grade every completed game; show the group's ATS record (season + weekly) and each person's individual accuracy record, side by side.

7. Explicitly out of scope

- Playoffs — regular season (Weeks 1-18) only.
- Per-user login/authentication beyond a single shared passcode.
- Payment/money-tracking of any kind — this is just about tracking picks and who's right.

8. Open decisions left to whoever builds this

- Hosting/backend architecture (a full custom web app vs. something lighter-weight) — no strong preference has been locked in; whatever is simplest to stand up and keep running reliably for a low-traffic, 3-person weekly-use app is fine.
- What happens if the same week's PDF gets uploaded twice (overwrite vs. block) — recommend: allow overwrite, but show a warning/diff first.
- Whether picks should hard-lock once a game starts, or stay editable indefinitely — recommend: don't over-engineer this for three trusted friends; editable indefinitely is fine for a first version.